

CST-407 Milestone 5 Implement Network Security with SSL

Alex M. Frear

College of Science, Engineering, and Technology, Grand Canyon University

Course Number: CST-407

Professor: Dr. Melody White

09/10/2025

Table of Contents

Step 1 — Demonstrate clear-text credential capture over HTTP using Wireshark	3
1.1 — Wireshark capture started (loopback interface)	3
1.2 — HTTP POST with credentials visible.....	4
Step 2 — Generate self-signed certificate and configure Spring Boot for HTTPS.....	5
2.1 — Keystore created.....	5
2.2 — Keystore in resources.....	6
2.3 — HTTPS properties configured.....	7
2.4 — Server started on 8443.....	8
2.5 — HTTPS page loads with cert warning (expected)	9
Step 3 — Configure HTTP connector and enforce automatic redirect to HTTPS	10
3.1 — HTTP→HTTPS redirect config class added.....	10
3.2 — HTTP port declared	11
3.3 — Security requires HTTPS	12
3.4 — Tomcat listening on 8080 and 8443	13
3.5 — Browser auto-redirected to HTTPS	14
3.6 — Network shows HTTP→HTTPS redirect.....	15
Step 4 — Capture HTTPS login and verify credentials are unreadable (TLS in Wireshark)	16
4.1 — Wireshark capture running (loopback)	16
4.2a — ClientHello shows SNI=localhost	17
4.2b — ServerHello shows TLS version and cipher.....	18
4.2c — Browser shows certificate CN=localhost	19
4.3 — TLS Application Data (encrypted payload).....	20
4.4 — Follow TCP Stream (encrypted, unreadable)	21
Step 5 — Verify HSTS response header and confirm HTTP→HTTPS redirect.....	22
5.1 — HSTS header present.....	22
5.2 — curl shows HSTS header	23
5.3 — HTTP request returns redirect to HTTPS	24
Summary of Key Points	25

Step 1 — Demonstrate clear-text credential capture over HTTP using Wireshark

1.1 — Wireshark capture started (loopback interface)

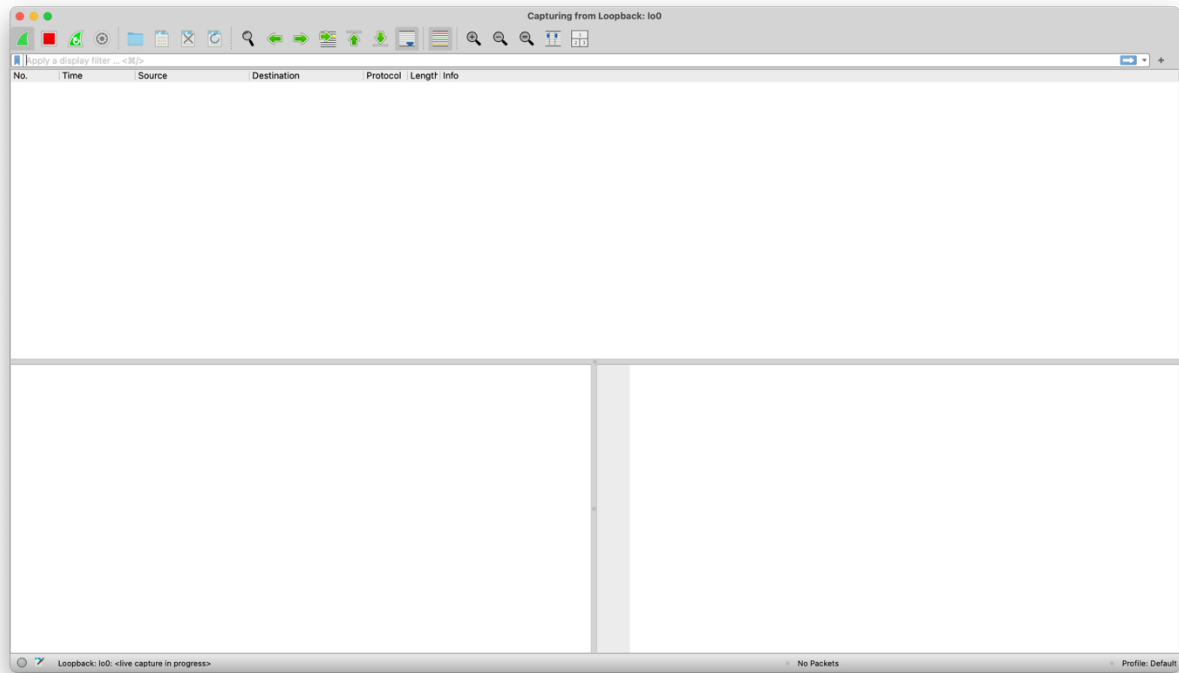


Figure 1 Shows Wireshark capturing on the local loopback interface so localhost traffic will be recorded.

1.2 — HTTP POST with credentials visible

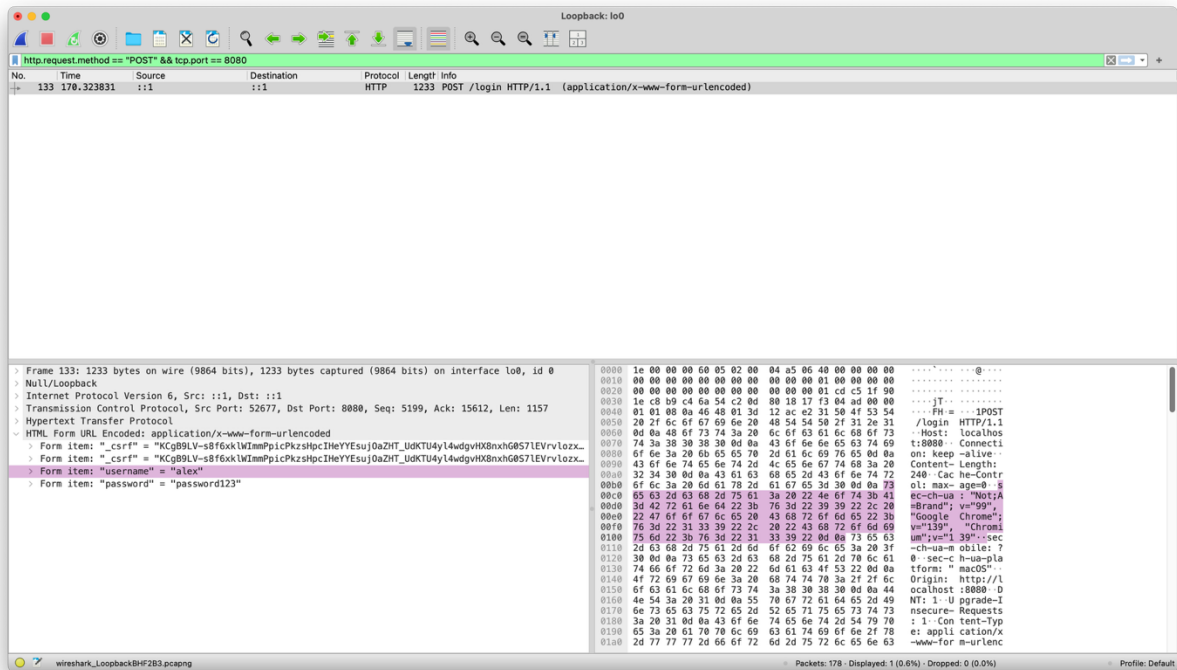
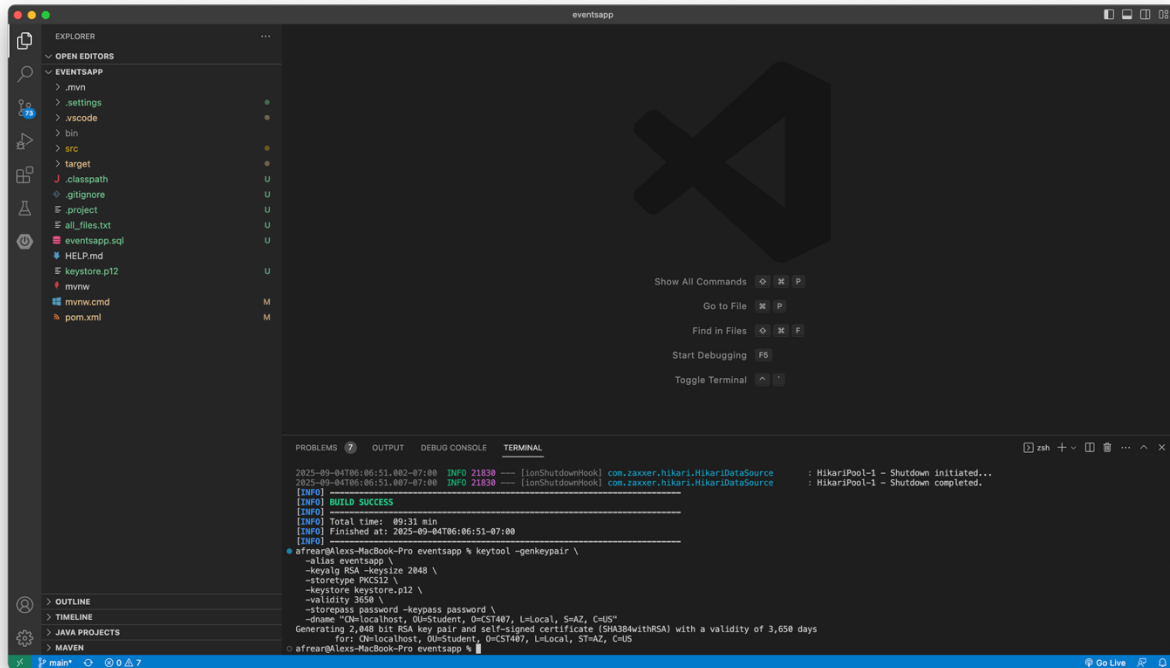


Figure 2 Displays the unencrypted username and password fields in the HTTP POST to /login, proving HTTP sends credentials in clear text.

Step 2 — Generate self-signed certificate and configure Spring Boot for HTTPS

2.1 — Keystore created



```
2025-09-04T06:06:51:06Z-07:00 INFO 21838 [onShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown initiated...
2025-09-04T06:06:51:07:00 INFO 21838 [onShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown completed.
[INFO] BUILD SUCCESS
[INFO] Total time: 09:31 min
[INFO] Finished at: 2025-09-04T06:06:51-07:00
[INFO]
afrear@Alexs-MacBook-Pro eventsapp % keytool -genkeypair \
  -alias eventsapp \
  -keyalg RSA -keysize 2048 \
  -storetype PKCS12 \
  -keystore keystore.p12 \
  -validity 3650 \
  -storepass password -keypass password \
  -name "CN=localhost, OU=Student, O=CS1487, L=Local, S=AZ, C=US"
Generating 2048 bit RSA key pair and self-signed certificate (SHA256withRSA) with a validity of 3,650 days
for: CN=localhost, OU=Student, O=CS1487, L=Local, S=AZ, C=US
afrear@Alexs-MacBook-Pro eventsapp %
```

Figure 3 Terminal confirms creation of `keystore.p12` with alias `eventsapp`, which will be used to enable HTTPS.

2.2 — Keystore in resources

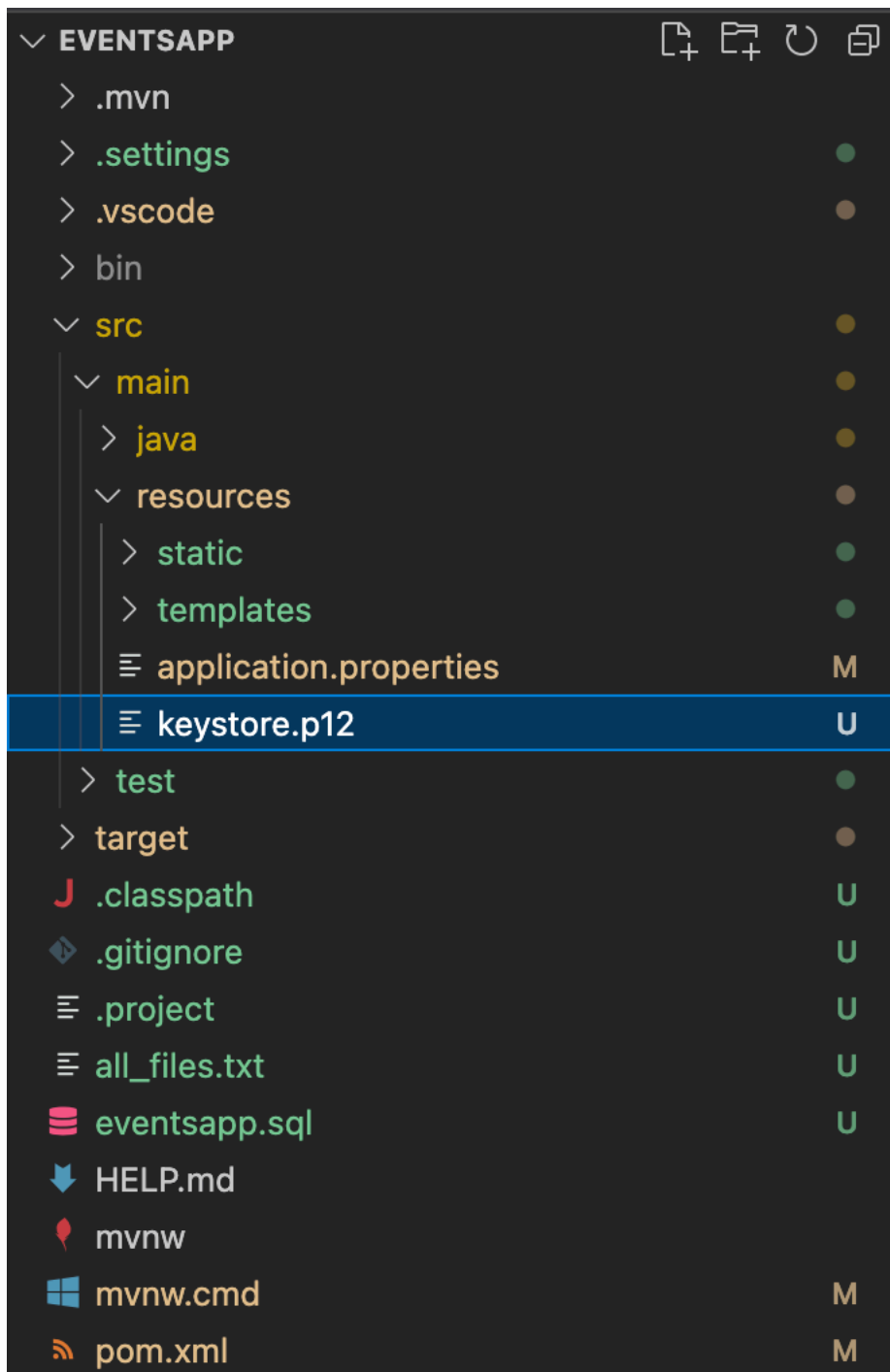
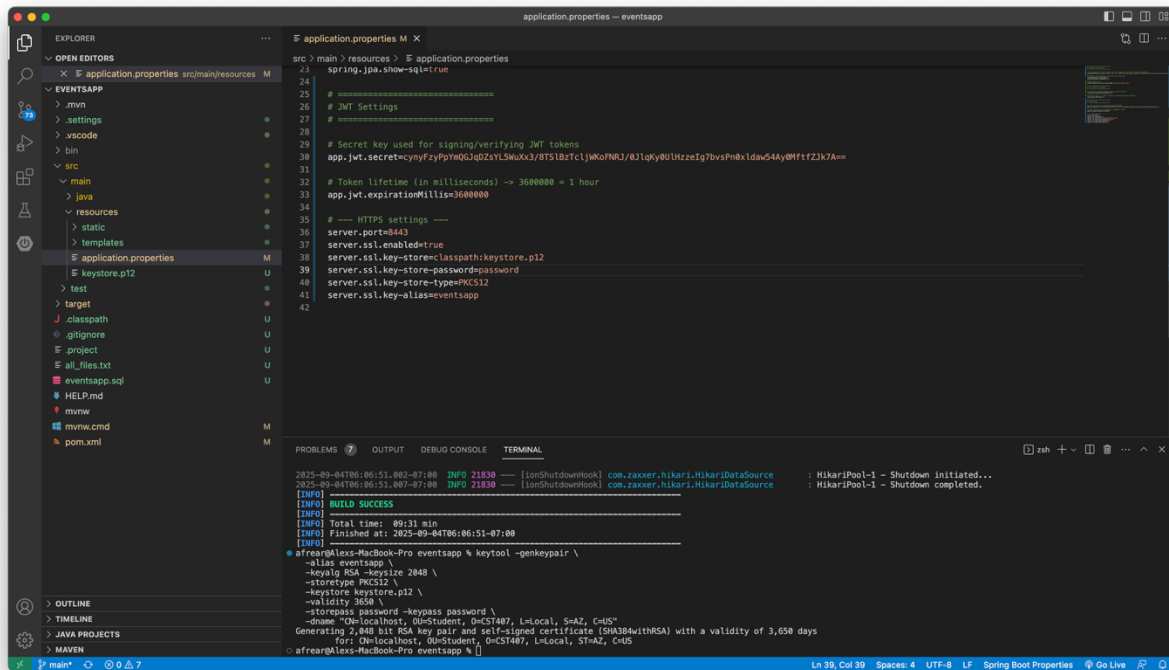


Figure 4 Shows `keystore.p12` located under `src/main/resources`, ensuring Spring Boot can load it at runtime.

2.3 — HTTPS properties configured



The screenshot shows a VS Code editor with a project named 'eventsapp'. The Explorer sidebar on the left shows the file structure, including 'application.properties' under 'resources'. The main editor displays the content of 'application.properties' with the following configuration:

```
src > main > resources > application.properties
23 spring.jpa.show-sql=true
24
25 # =====
26 # JWT Settings
27 # =====
28
29 # Secret key used for signing/verifying JWT tokens
30 app.jwt.secret=cmyfzyPpnoQ3q2x1t5w0X3/8T518t7c1jWofNRJ/03lqhy0UHzzeIg7bvsPn8xldw044y0Hftf23k7A==
31
32 # Token lifetime (in milliseconds) -> 3600000 = 1 hour
33 app.jwt.expirationMillis=3600000
34
35 # --- HTTPS settings ---
36 server.port=8443
37 server.ssl.enabled=true
38 server.ssl.key-store-classpath=keystore.p12
39 server.ssl.key-store-password=password
40 server.ssl.key-store-type=PKCS12
41 server.ssl.key-alias=eventsapp
42
```

The bottom panel shows the 'TERMINAL' output, which includes the following log messages:

```
2025-09-04T06:06:51:00Z-07:00 INFO 21838 --- [onShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown initiated...
2025-09-04T06:06:51:00Z-07:00 INFO 21838 --- [onShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown completed.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 00:31 min
[INFO] Finished at: 2025-09-04T06:06:51:00Z
[INFO] -----
afrearg@Alex-MacBook-Pro eventsapp % keytool -genkeypair \
-alias eventsapp \
-keyalg RSA -keysize 2048 \
-storetype PKCS12 \
-keystore keystore.p12 \
-validity 3650 \
-storepass password -keypass password \
-dname "CN=localhost, OU=Student, O=CST487, L=Local, S=AZ, C=US"
Generating 2,048 bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 3,650 days
for: CN=localhost, OU=Student, O=CST487, L=Local, ST=AZ, C=US
afrearg@Alex-MacBook-Pro eventsapp % |
```

Figure 5 Shows Spring Boot SSL properties pointing to the keystore and moving the app to port 8443.

2.4 — Server started on 8443

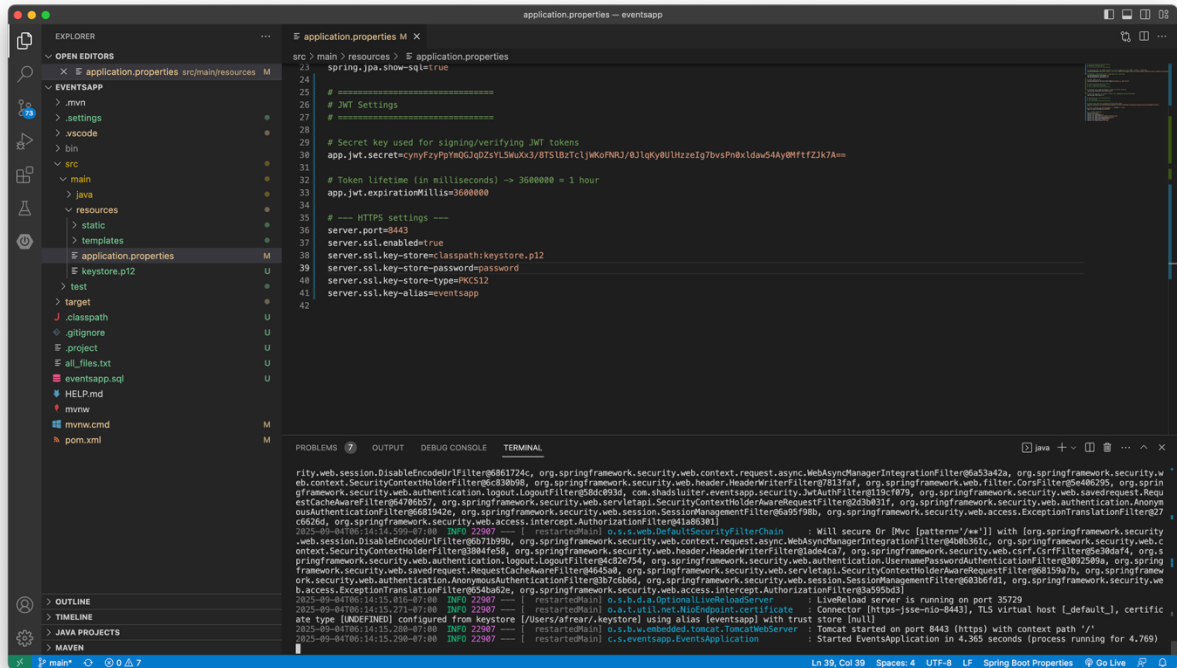


Figure 6 Confirms the application is now listening on HTTPS port 8443.

2.5 — HTTPS page loads with cert warning (expected)

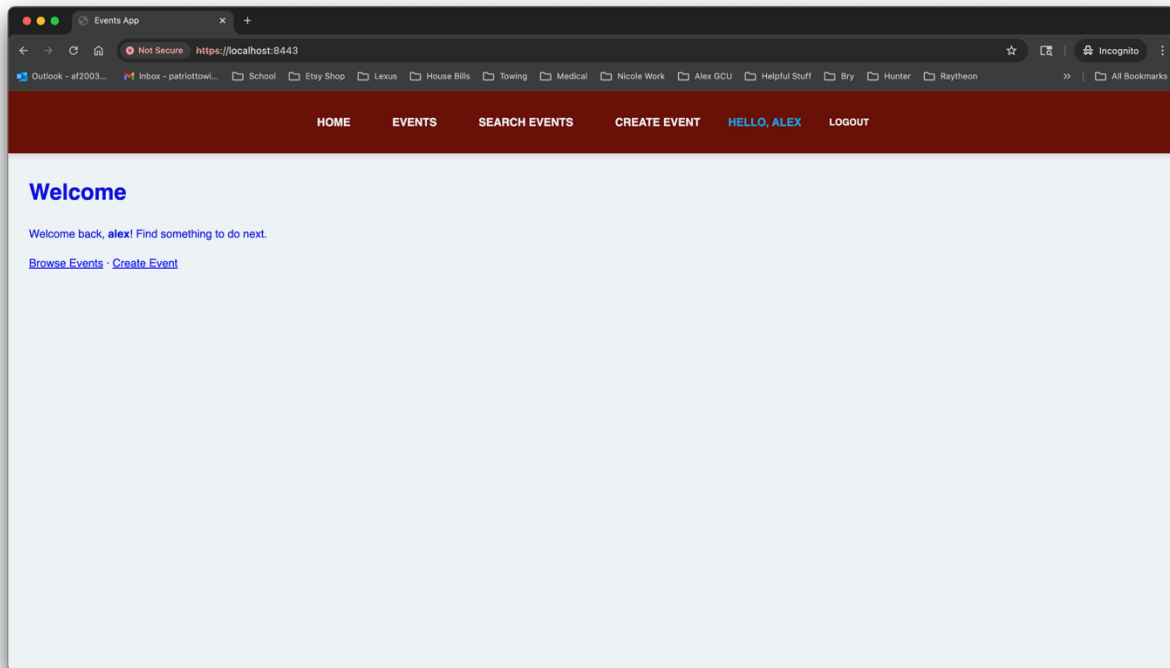


Figure 7 Shows the self-signed certificate warning; proceeding confirms the app serves content over HTTPS.

Step 3 — Configure HTTP connector and enforce automatic redirect to HTTPS

3.1 — HTTP→HTTPS redirect config class added

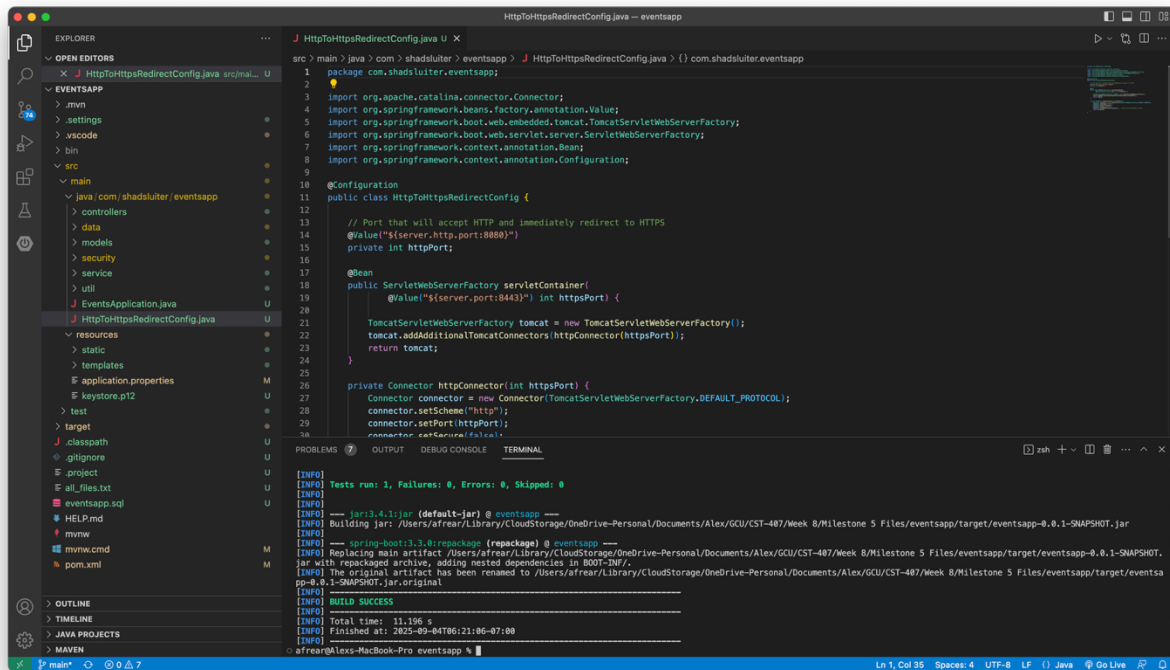


Figure 8 Shows the `HttpToHttpsRedirectConfig` bean that adds an HTTP connector on port 8080 and redirects to 8443.

3.2 — HTTP port declared

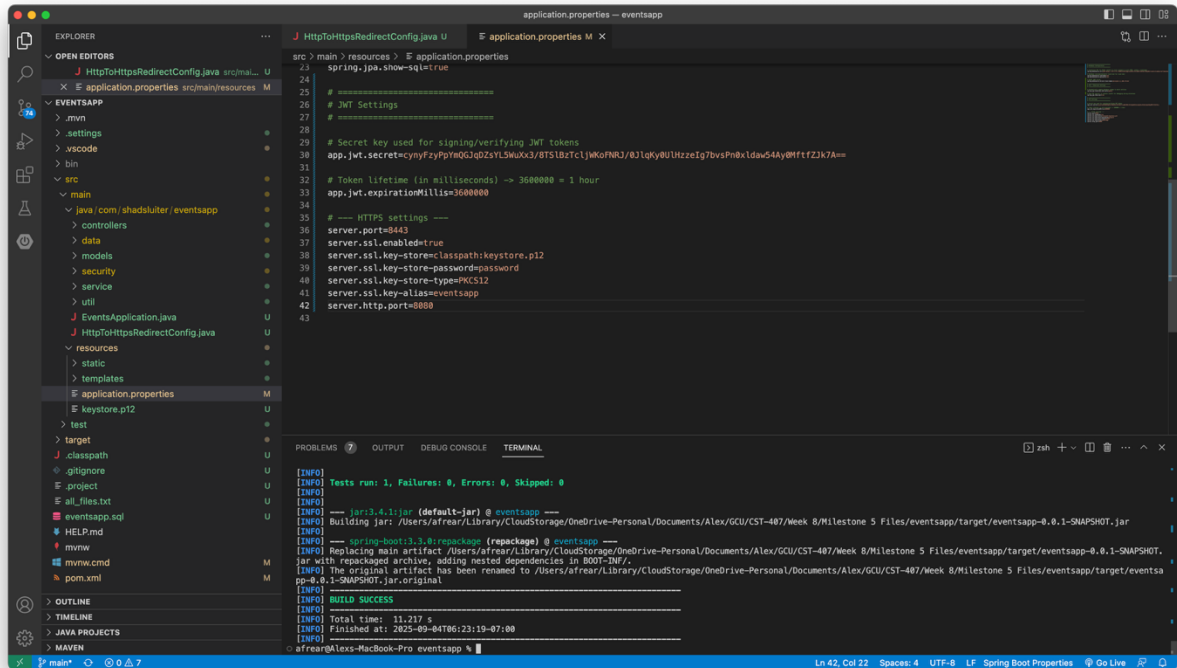


Figure 9 Confirms `server.http.port=8080` so Tomcat exposes an HTTP listener that only redirects to HTTPS.

3.3 — Security requires HTTPS

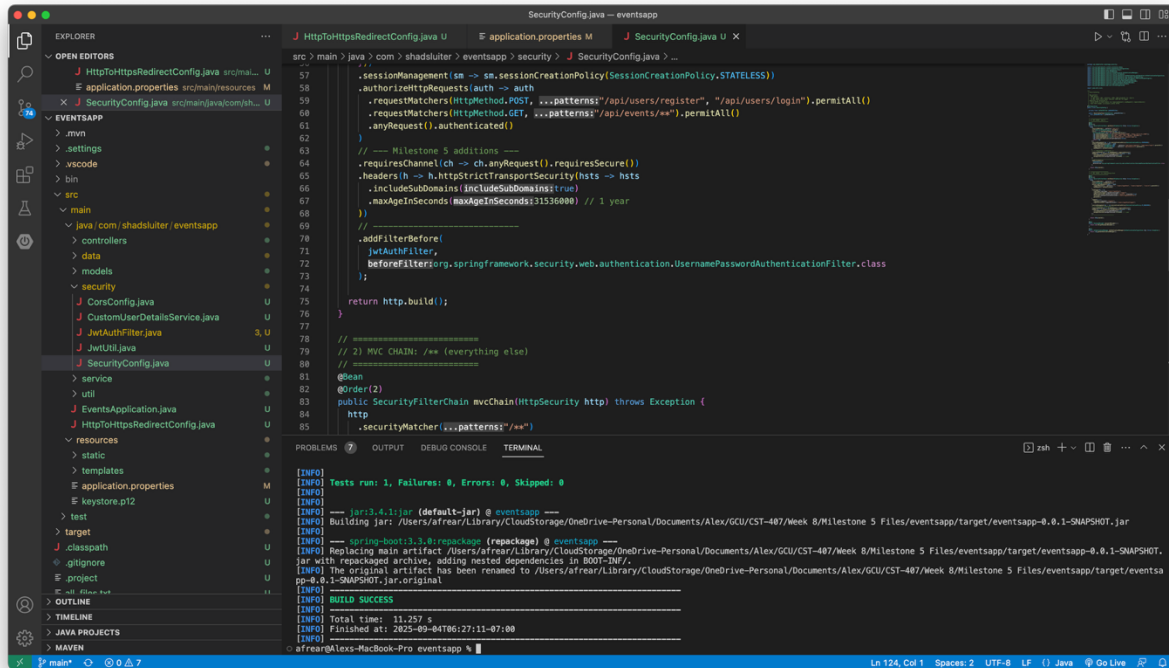


Figure 10 Shows Spring Security requiring secure channels and emitting HSTS, ensuring all routes are served via HTTPS.

3.4 — Tomcat listening on 8080 and 8443

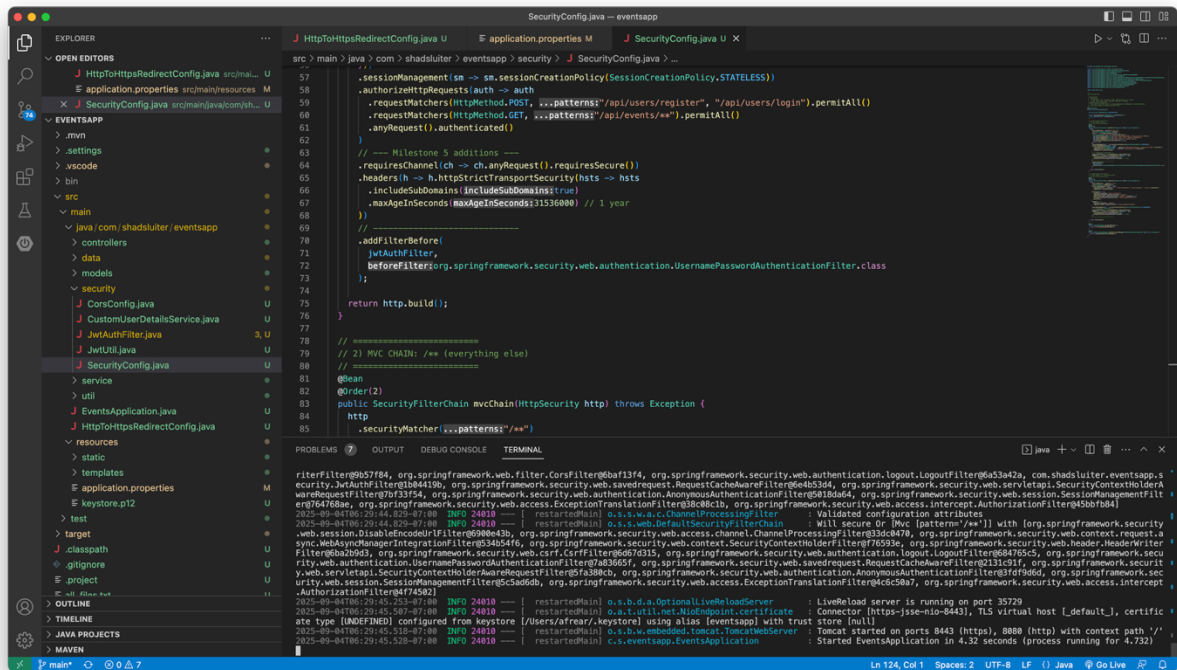


Figure 11 Confirms both connectors are active: HTTP (8080) for redirect and HTTPS (8443) for secure traffic.

3.5 — Browser auto-redirected to HTTPS

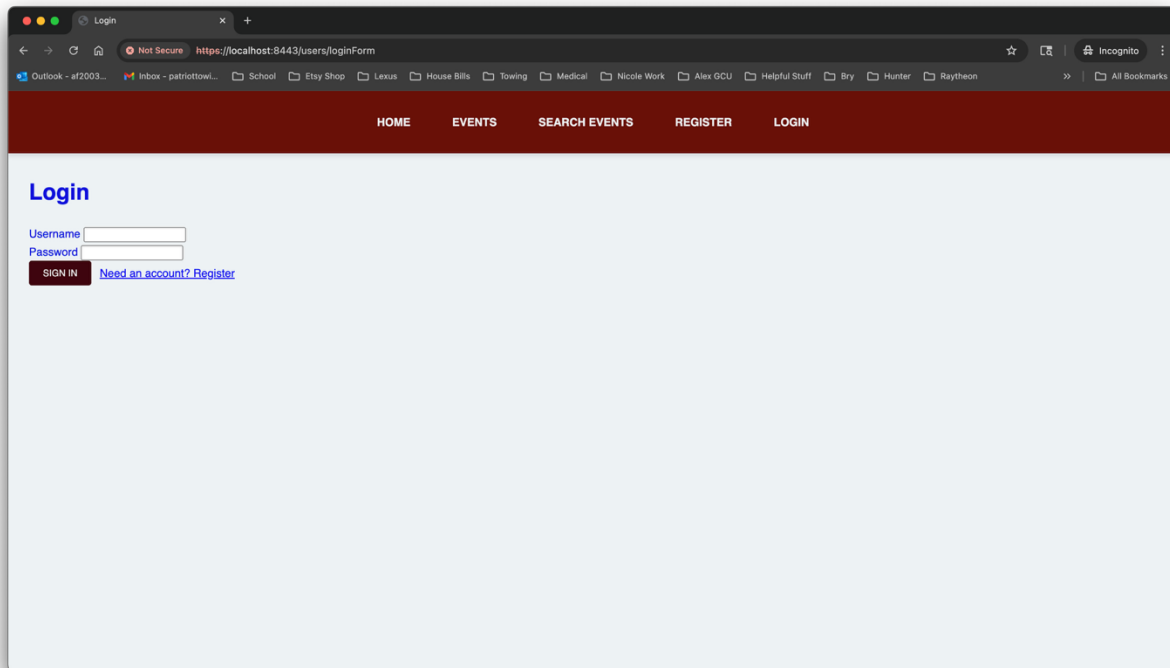


Figure 12 Location bar shows the final HTTPS URL after requesting HTTP, proving the redirect.

3.6 — Network shows HTTP→HTTPS redirect

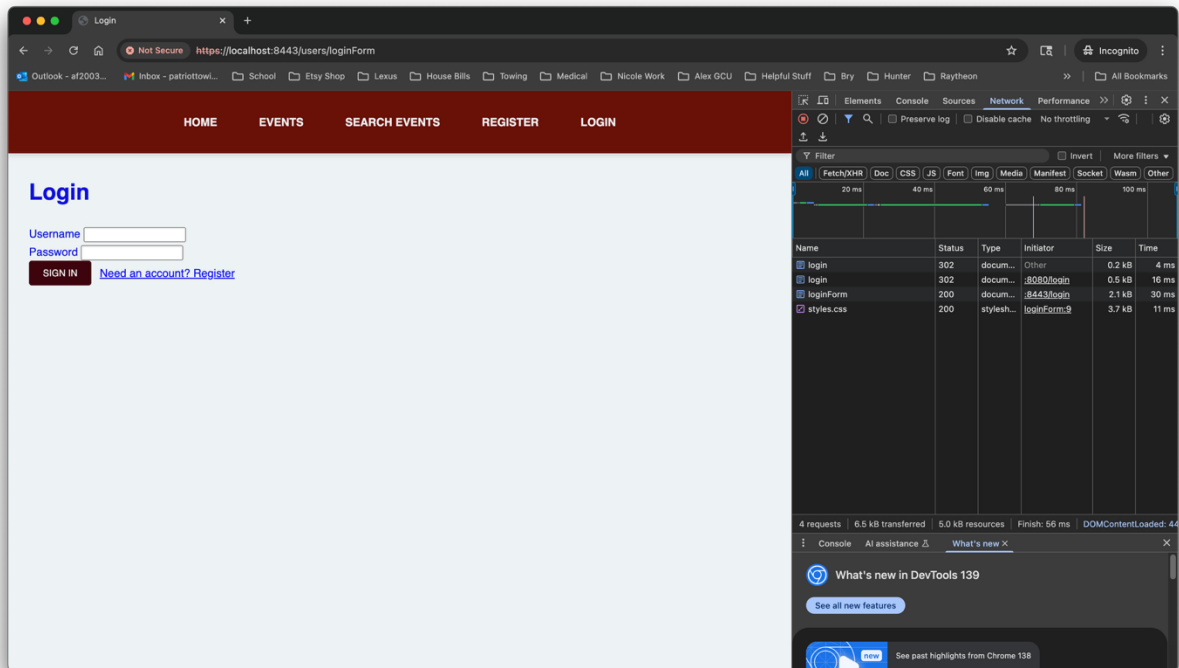


Figure 13 Network panel displays the redirect status and Location header to `https://localhost:8443/login`, proving enforcement at the server.

Step 4 — Capture HTTPS login and verify credentials are unreadable (TLS in Wireshark)

4.1 — Wireshark capture running (loopback)

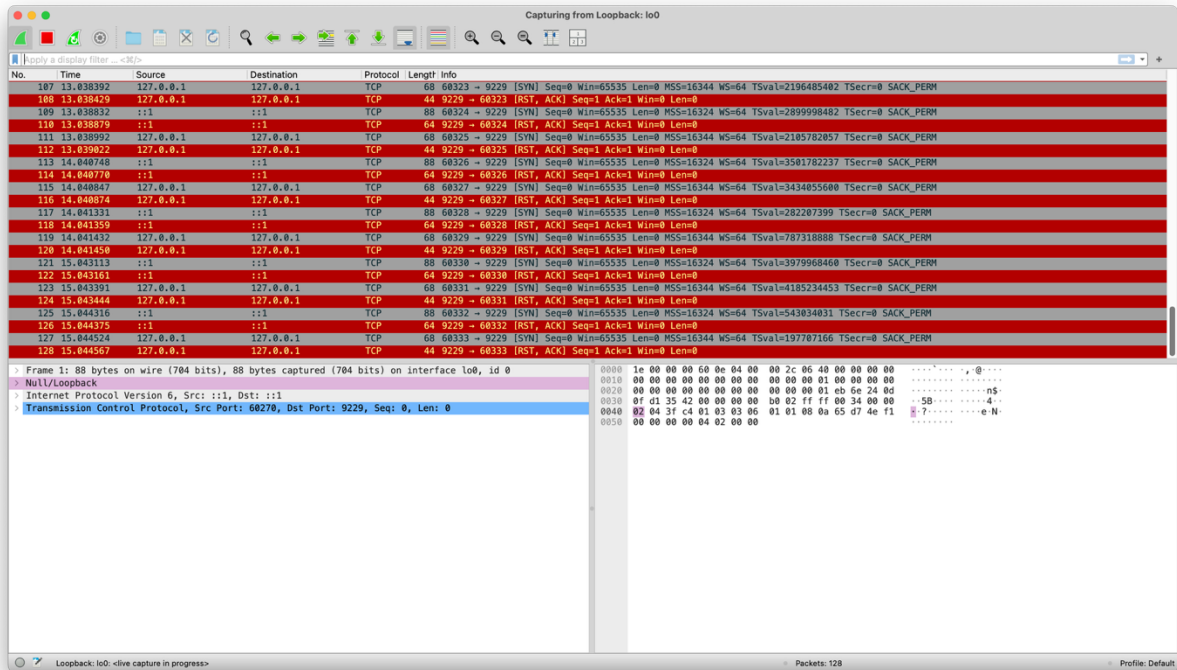


Figure 14 Shows Wireshark actively capturing on the local loopback to record TLS traffic to localhost.

4.2a — ClientHello shows SNI=localhost

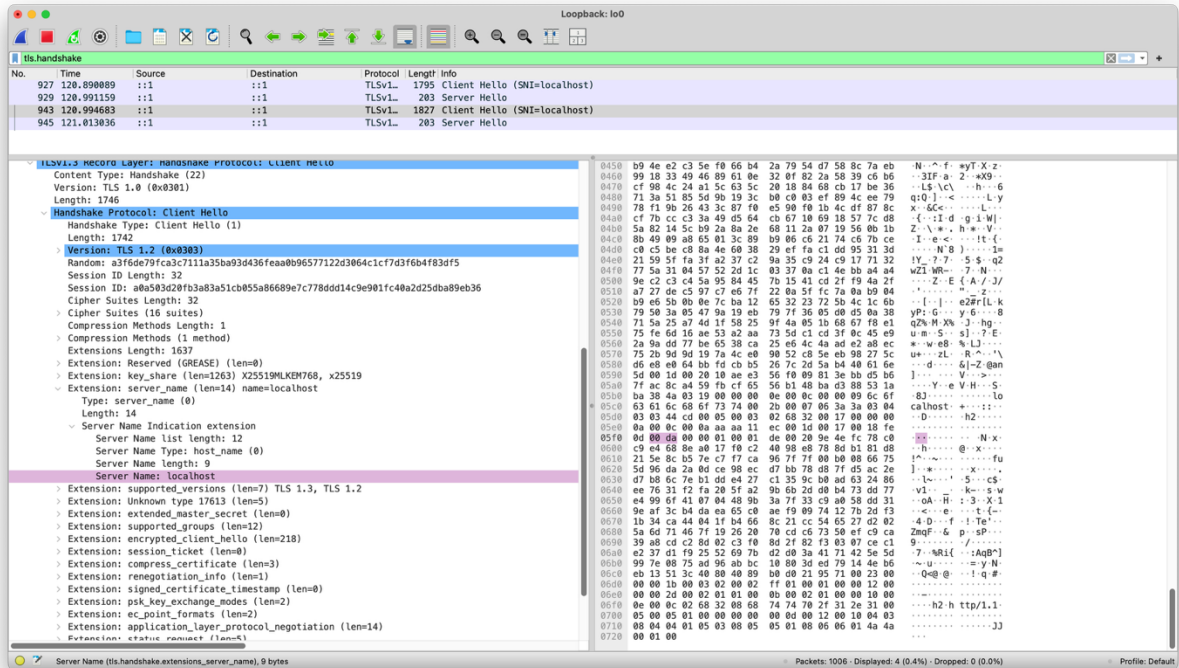


Figure 15 Displays the Client Hello extension with Server Name: localhost, proving the client indicated the host.

4.2b — ServerHello shows TLS version and cipher

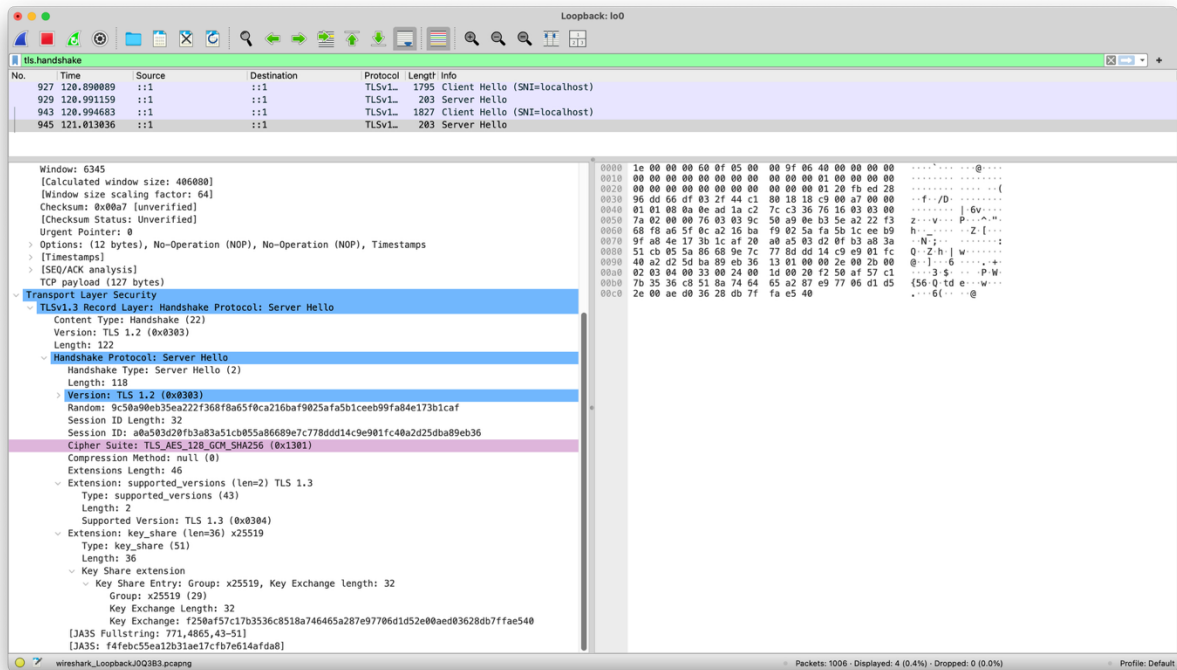


Figure 16 Shows the negotiated TLS version and selected cipher suite for the session.

4.2c — Browser shows certificate CN=localhost

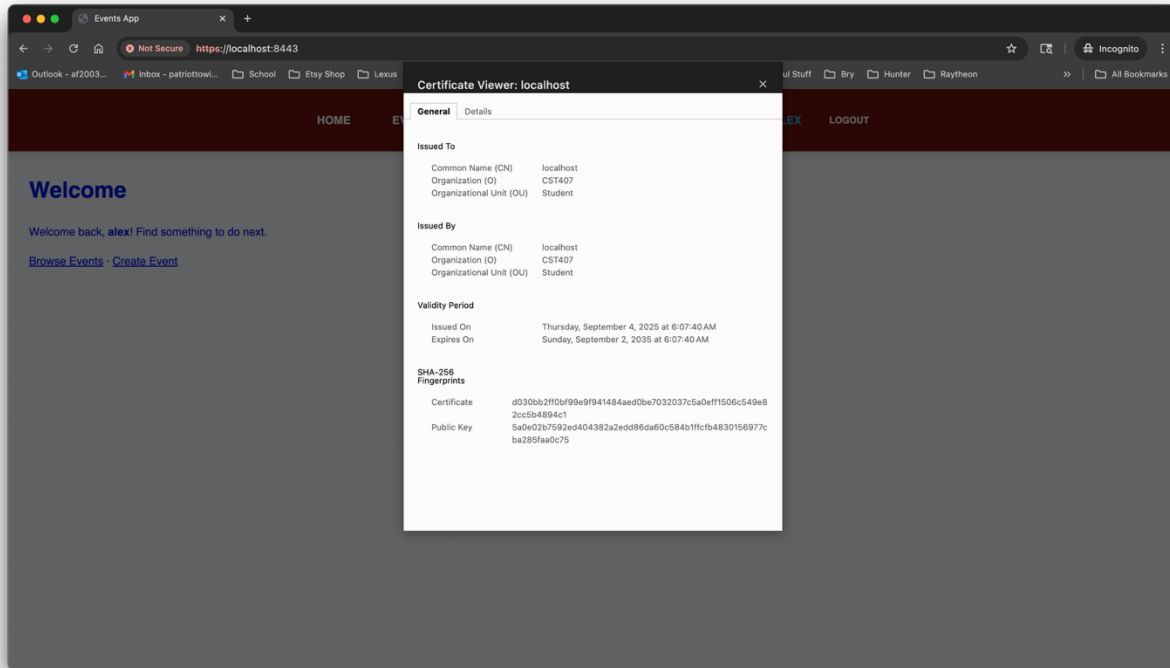


Figure 17 The browser certificate viewer shows Subject **CN=localhost**, matching the self-signed cert I created.

4.3 — TLS Application Data (encrypted payload)

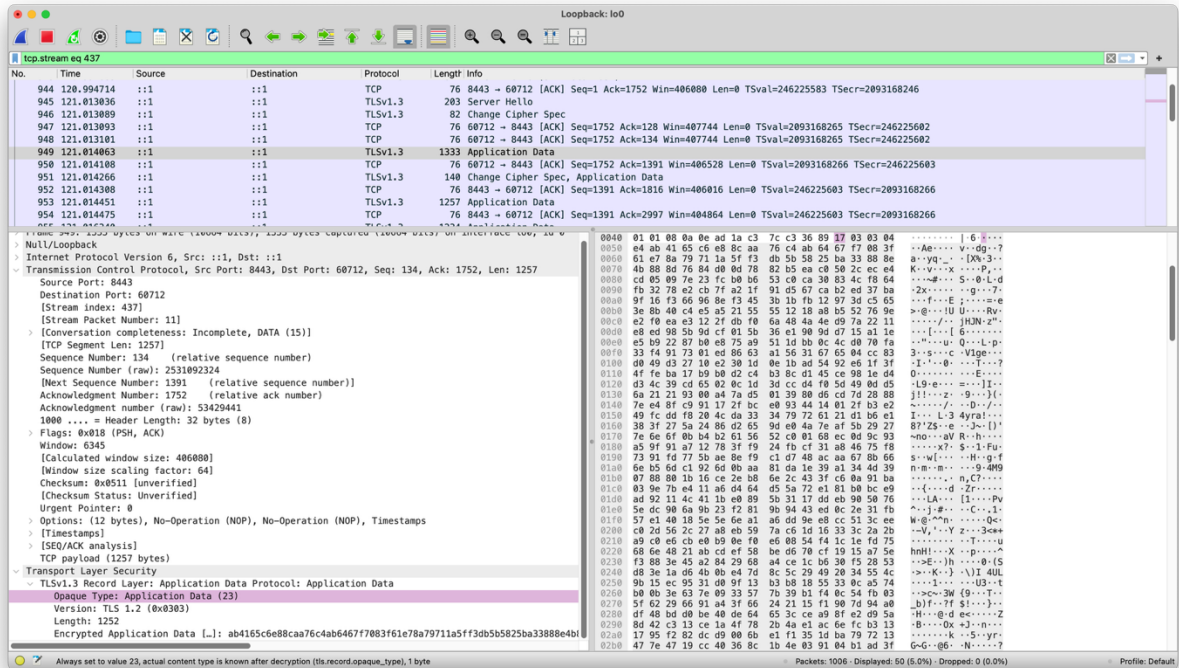


Figure 18 Packet details show the request carried as **TLS Application Data** with only encrypted bytes, proving the login POST is not readable on the wire.

4.4 — Follow TCP Stream (encrypted, unreadable)

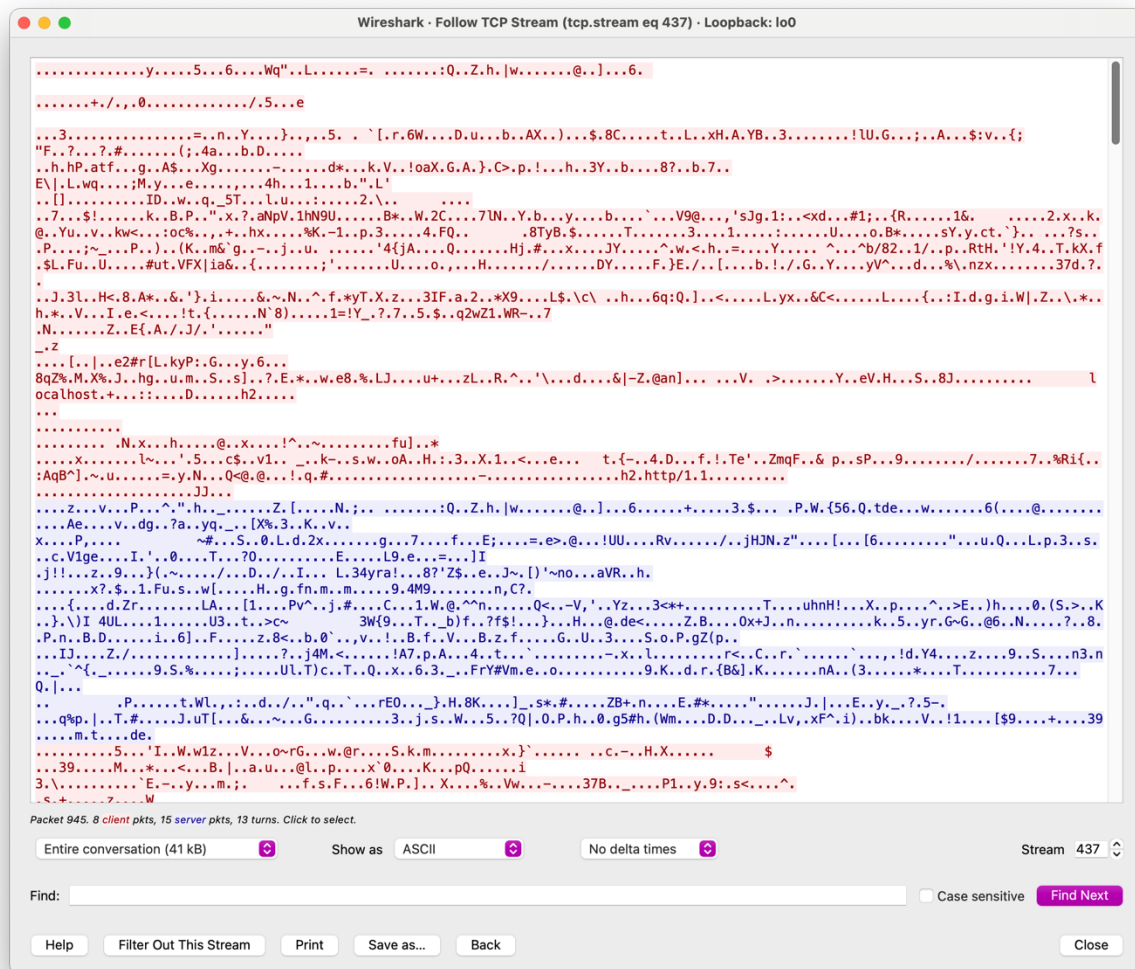


Figure 19 The TLS stream contents are unreadable binary/garbled text; the HTTP form fields (username/password) are not visible because the payload is encrypted.

Step 5 — Verify HSTS response header and confirm HTTP→HTTPS redirect

5.1 — HSTS header present

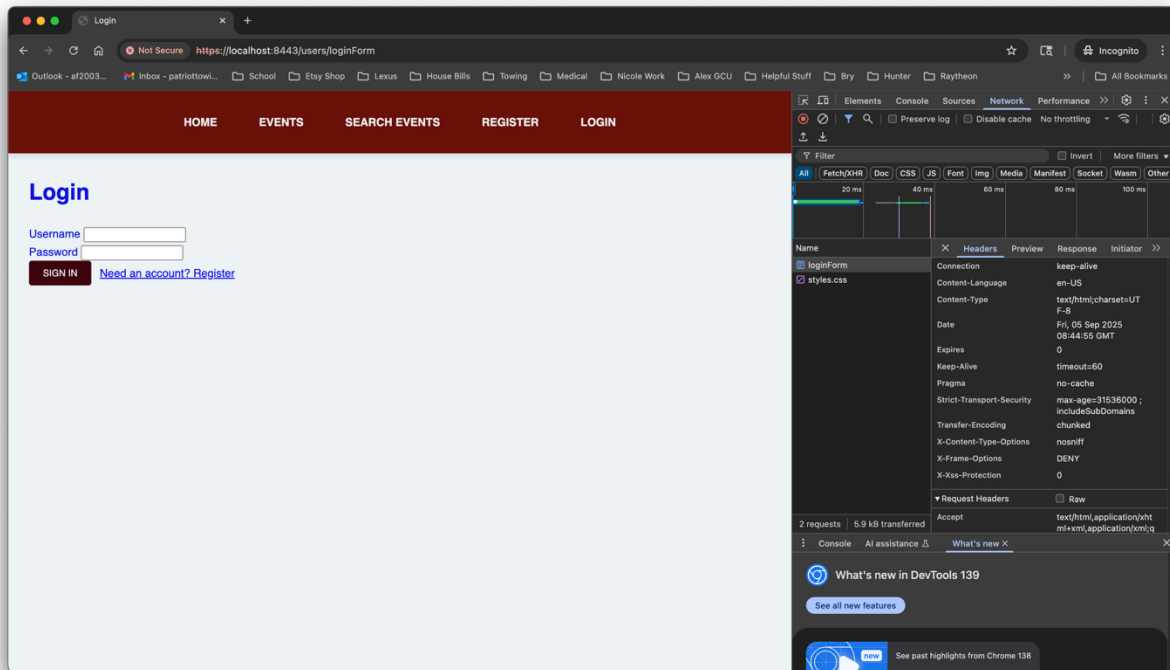
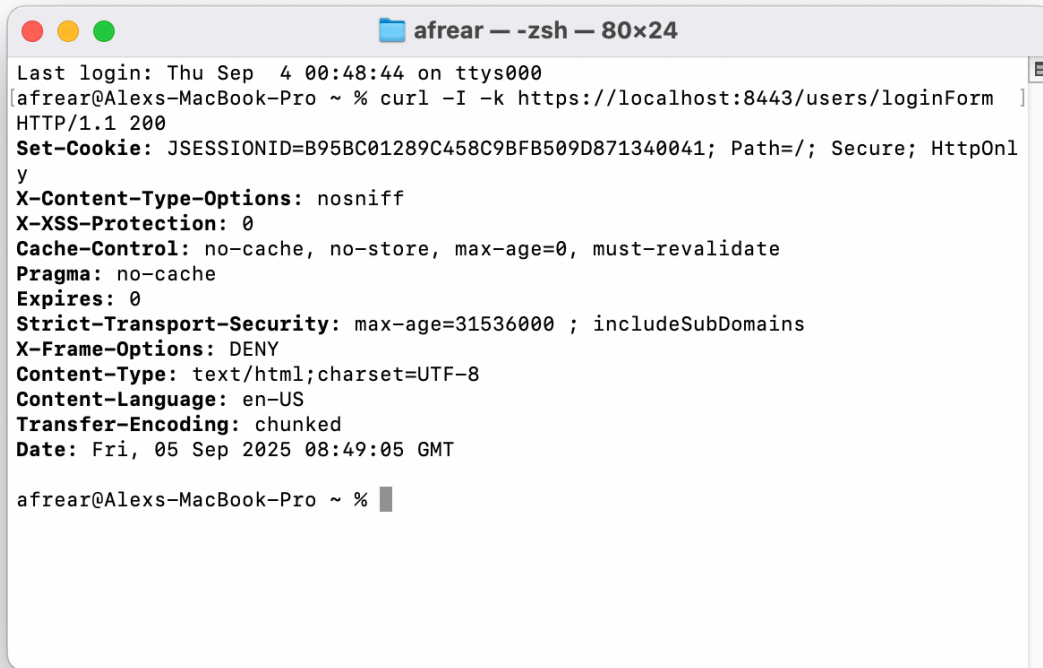


Figure 20 Response headers include `strict-transport-security`, proving the server instructs browsers to prefer HTTPS.

5.2 — curl shows HSTS header

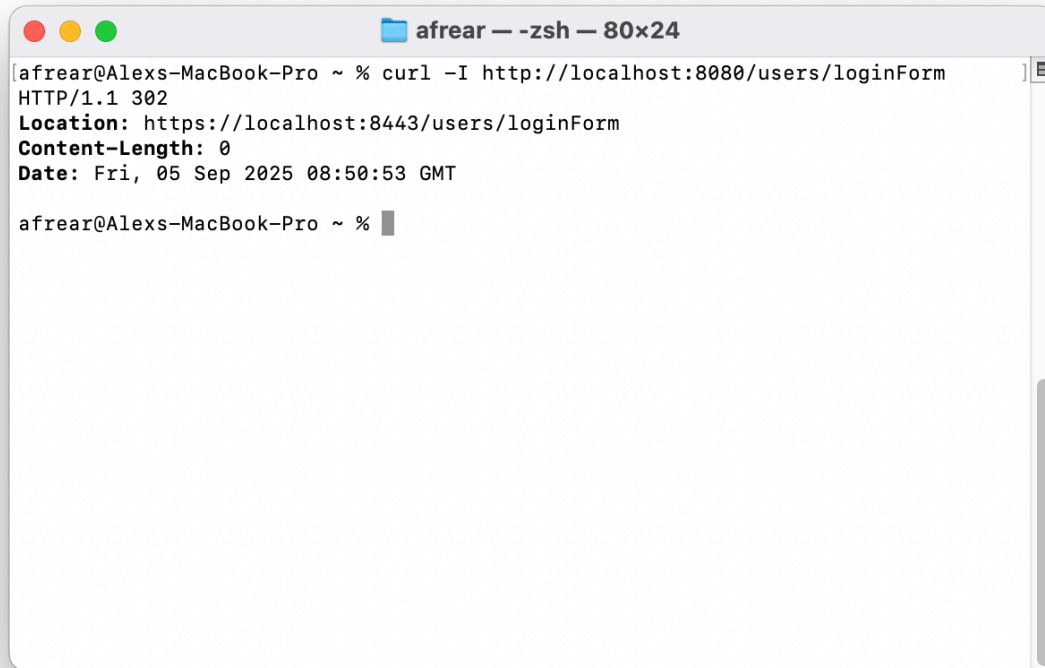
A terminal window titled "afrear — -zsh — 80x24" with standard macOS window controls (red, yellow, green buttons). The terminal shows the output of a curl command. The output includes a status line, a set-cookie header, and several security-related headers, including Strict-Transport-Security.

```
Last login: Thu Sep  4 00:48:44 on ttys000
[afrear@Alexs-MacBook-Pro ~ % curl -I -k https://localhost:8443/users/loginForm ]
HTTP/1.1 200
Set-Cookie: JSESSIONID=B95BC01289C458C9BFB509D871340041; Path=/; Secure; HttpOnly
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
Strict-Transport-Security: max-age=31536000 ; includeSubDomains
X-Frame-Options: DENY
Content-Type: text/html; charset=UTF-8
Content-Language: en-US
Transfer-Encoding: chunked
Date: Fri, 05 Sep 2025 08:49:05 GMT

afrear@Alexs-MacBook-Pro ~ %
```

Figure 21 Terminal output displays the `strict-transport-security` header returned by the HTTPS endpoint.

5.3 — HTTP request returns redirect to HTTPS



```
afrear@Alexs-MacBook-Pro ~ % curl -I http://localhost:8080/users/loginForm
HTTP/1.1 302
Location: https://localhost:8443/users/loginForm
Content-Length: 0
Date: Fri, 05 Sep 2025 08:50:53 GMT

afrear@Alexs-MacBook-Pro ~ %
```

Figure 22 Shows HTTP response status and Location header pointing to the HTTPS URL, confirming automatic redirection.

Summary of Key Points

For Milestone 5, I first captured an HTTP login with Wireshark and showed that the POST body revealed my username and password in clear text. I then generated a self-signed certificate, enabled HTTPS on port 8443 in Spring Boot, and added an additional Tomcat HTTP connector on port 8080 strictly for redirection. In Spring Security, I required secure channels on both the API and MVC filter chains and sent an HSTS header so browsers prefer HTTPS. I verified the browser was redirected from HTTP to HTTPS, and I confirmed in Wireshark that the TLS handshake completed and all subsequent packets were TLS Application Data rather than readable HTTP. Following the TCP stream showed only encrypted bytes, meaning credentials were no longer visible on the wire. Finally, I proved the `strict-transport-security` header was present using both DevTools and `curl`, completing the end-to-end demonstration that the application now enforces HTTPS and protects login credentials in transit.